

MEMORY MANAGEMENT UNIT (MMU) DESIGN USING VERILOG HDL

Abstract

A Memory Management Unit (MMU) is responsible for translating Virtual Addresses generated by a processor into Physical Addresses used by memory hardware. In addition to address translation, the MMU provides memory protection and access control mechanisms. This project implements a Verilog HDL based MMU capable of performing page table lookup, valid bit checking, permission verification, page fault generation, and protection fault detection.

Chapter 1: Introduction

Modern processors rely on virtual memory systems to provide memory isolation, security, and efficient memory utilization. The MMU acts as the hardware interface between the processor and memory subsystem.

The primary responsibility of the MMU is to convert Virtual Addresses into Physical Addresses while enforcing memory access permissions.

Chapter 2: Objectives

The objectives of this project are:

1. Design a Verilog-based MMU.
 2. Implement Virtual-to-Physical Address Translation.
 3. Implement Page Table Lookup.
 4. Detect Invalid Pages.
 5. Detect Access Permission Violations.
 6. Verify functionality using simulation.
-

Chapter 3: MMU Architecture

The architecture consists of:

- Virtual Address Input

- Page Number Extraction
- Offset Extraction
- Page Table Lookup
- Valid Bit Verification
- Permission Verification
- Physical Address Generation
- Fault Detection Logic

Architecture Flow:

Virtual Address ↓ Page Table Lookup ↓ Valid Bit Check ↓ Permission Check ↓ Physical Address

Chapter 4: Address Translation

The 16-bit Virtual Address is divided into:

Page Number = $VA[15:8]$

Offset = $VA[7:0]$

The page number indexes the page table.

The selected entry provides:

- Frame Number
- Valid Bit
- Read Permission
- Write Permission

The frame number is concatenated with the offset to form the Physical Address.

Chapter 5: Page Table Design

Each Page Table Entry contains:

Valid Bit

Read Permission

Write Permission

Frame Number

Example:

Page 4:

Valid = 1

Read = 1

Write = 0

Frame = 20

Result:

Read operations are allowed while write operations generate a protection fault.

Chapter 6: RTL Design

The MMU module performs:

1. Page Number Extraction
2. Page Table Lookup
3. Valid Bit Verification
4. Permission Verification
5. Physical Address Generation
6. Fault Signal Generation

Outputs:

- physical_addr

- page_fault
 - protection_fault
 - translation_valid
-

Chapter 7: Verification

Verification was performed using a dedicated Verilog Testbench.

Test Cases:

1. Valid Translation
2. Invalid Page Access
3. Write Protected Page Access
4. Boundary Address Access

All test cases passed successfully.

Chapter 8: Simulation Results

Observed Results:

Valid Translation: translation_valid = 1

Invalid Page: page_fault = 1

Write Protected Page: protection_fault = 1

The waveform confirms correct functionality of the MMU design.

[Insert Waveform Screenshots]

Chapter 9: FPGA Implementation

The design can be synthesized using Vivado.

Implementation Flow:

RTL Design ↓ Synthesis ↓ Implementation ↓ Bitstream Generation

Switches may be used as Virtual Address inputs while LEDs display Physical Address and Fault Signals.

Chapter 10: Conclusion

A Verilog HDL based Memory Management Unit was successfully designed and verified. The MMU translated Virtual Addresses into Physical Addresses using a Page Table based architecture. Memory protection was implemented using Valid Bits and Permission Bits. Simulation results confirmed successful address translation, page fault generation, and protection fault detection. The project demonstrates practical concepts from Computer Architecture, Digital Design, and VLSI System Design.